팀2 — 공장 에너지 피크관리 · 피크위험 구간 예측

K-뉴딜 아카데미 스마트팩토리 캡스톤 · 3~6회차 통합 실습

데이터	분석단위	규모	기간
`team2_energy.csv`	30분 단위 공장 에너지 운영기록 1건	1,800행 × 24열	2026년 7월 1일 ~ 8월 7일 (30분 간격)

이 노트북에서 하는 일

데이터 탐색 → 시각화 → 전처리 → 머신러닝 모델 3종 → 성능평가 → 특성중요도

현장 상황

가상의 **가전 제조공장**입니다. 같은 생산량을 만드는 시간대인데도 전력사용량과 **최대수요전력(peak demand)** 이 구간마다 다르게 나타납니다. 여러 생산라인과 공조·압축공기 설비가 동시에 돌면 전력 피크가 생기고, 전기요금이 올라갑니다.

여기서 **최대수요전력**이라는 개념을 꼭 이해해야 합니다. 산업용 전기요금은 "쓴 양(kWh)"에 대한 **사용요금**과, "한 달 중 가장 세게 쓴 순간(kW)"에 대한 **기본요금**으로 나뉩니다. 후자는 **단 한 번의 피크가 그 달(심하면 1년) 요금의 기준**이 됩니다. 그래서 에너지 담당자는 총 사용량보다 **"피크를 만들지 않는 운전"** 에 집중합니다.

담당자는 다음 날 생산계획을 확정하면서, **생산목표는 지키되 피크위험 시간대를 피해 설비 가동을 재배치**해야 합니다.

핵심 질문

> **생산손실을 최소화하면서 전력 피크를 낮추려면, 어느 구간이 위험하고 무엇을 옮겨야 하는가?**

판단 시점

> **다음 구간(또는 다음 날) 생산계획을 확정하는 시점 (계획수량·가동라인·요금시간대·기상예보는 알고, 실제 전력은 아직 모름)**

판단 시점이 중요한 이유: **그 시점에 알 수 없는 값**을 입력으로 쓰면 시험 답안지를 미리 본 것과 같아집니다(**데이터 누출**). 아래 8절에서 직접 걸러냅니다.

---

1. 준비 — 라이브러리와 한글 폰트

도구	역할
`pandas` (pd)	표 데이터 처리 (엑셀에 해당)
`numpy` (np)	숫자 계산
`matplotlib` (plt) · `seaborn` (sns)	그래프
`scikit-learn`	머신러닝

아래 셀은 **한 번만** 실행하면 됩니다.

```python
# [준비 셀] 내용을 자세히 이해할 필요는 없습니다. 실행만 하세요.
# 라이브러리 불러오기 + 한글 폰트 설정
import os, sys, glob, warnings
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report, roc_curve,
                             roc_auc_score, recall_score, precision_score)
warnings.filterwarnings("ignore")
IN_COLAB = "google.colab" in sys.modules

# ------------------------------------------------------------------------
# 한글 폰트 설정
#   ※ 순서 주의: seaborn 스타일을 먼저 적용해야 한다.
#      sns.set_style() 이 글꼴 설정을 sans-serif 로 덮어쓰기 때문이다.
# ------------------------------------------------------------------------
sns.set_style("whitegrid")                      # ① 스타일 먼저

from matplotlib import font_manager as fm

KOREAN_FONTS = ["NanumGothic", "NanumBarunGothic", "Malgun Gothic", "AppleGothic",
                "Apple SD Gothic Neo", "Noto Sans CJK KR", "Noto Sans KR"]


def find_korean_font():
    """설치된 한글 폰트를 찾고, 없으면 Colab에서 나눔폰트를 설치한다."""
    have = {f.name for f in fm.fontManager.ttflist}
    for n in KOREAN_FONTS:
        if n in have:
            return n
    if IN_COLAB:                                # 나눔폰트 설치 후 다시 찾기
        print("한글 폰트를 설치하는 중입니다... (20초 정도 걸립니다)")
        os.system("apt-get install -y fonts-nanum > /dev/null 2>&1")
        os.system(f"{sys.executable} -m pip install -q koreanize-matplotlib")
        for path in glob.glob("/usr/share/fonts/truetype/nanum/Nanum*.ttf"):
            fm.fontManager.addfont(path)
        have = {f.name for f in fm.fontManager.ttflist}
        for n in KOREAN_FONTS:
            if n in have:
                return n
    return None

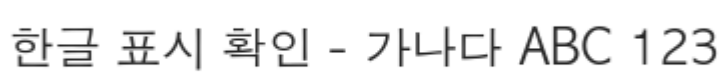

FONT = find_korean_font()                       # ② 폰트 나중에
if FONT:
    plt.rcParams["font.family"] = FONT
    plt.rcParams["font.sans-serif"] = [FONT] + list(plt.rcParams["font.sans-serif"])
plt.rcParams["axes.unicode_minus"] = False      # 음수 기호(-) 깨짐 방지
plt.rcParams["figure.figsize"] = (9, 4.5)
plt.rcParams["figure.dpi"] = 110

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

print("준비 완료 | pandas", pd.__version__, "| Colab:", IN_COLAB)
print("한글 폰트 :", FONT if FONT else "찾지 못함")
```

```
# ③ 자가진단 — 아래 그림에 한글이 보이면 정상입니다
plt.figure(figsize=(5, 0.9))
plt.text(0.5, 0.5, "한글 표시 확인 — 가나다 ABC 123",
         ha="center", va="center", fontsize=15)
plt.axis("off")
plt.show()
print("위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.")
```

```
준비 완료 | pandas 2.2.3 | Colab: False
한글 폰트 : AppleGothic
```




```
위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.
```

---

2. 데이터 가져오기

두 가지 방법이 있습니다. **① GitHub에서 바로 내려받기**(권장)가 안 되면 **② PC에서 업로드**를 쓰세요.

```
# ① GitHub에서 바로 내려받기
URL = "https://raw.githubusercontent.com/StillWork/capstone2610/main/team2_energy.csv"
df_raw = pd.read_csv(URL)

# ② PC에서 업로드 — ①이 안 되면 위 2줄을 지우고 아래 3줄의 주석을 푸세요
# from google.colab import files
# up = files.upload()
# df_raw = pd.read_csv(list(up.keys())[0], encoding="utf-8-sig")

print("크기(행, 열):", df_raw.shape)
df_raw.head(3)
```

```
크기(행, 열): (1800, 24)
```

	record_id	timestamp	day_type	shift	tariff_period	active_line_count	product_mix	production_qty	plar
0	EN-00001	2026-07-01 00:00:00	WEEKDAY	Day	OFF	2	MIX_A	99	
1	EN-00002	2026-07-01 00:30:00	WEEKDAY	Day	OFF	2	MIX_A	92	
2	EN-00003	2026-07-01 01:00:00	WEEKDAY	Day	OFF	2	MIX_B	97	

---

3. 데이터 훑어보기

새 데이터를 받으면 **크기 → 자료형·결측 → 요약통계** 순으로 봅니다. 요약통계에서는 **불가능한 값**(음수 수량, 범위를 벗어난 값)이 있는지 확인합니다.

```
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1800 entries, 0 to 1799
Data columns (total 24 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   record_id                     1800 non-null   object 
 1   timestamp                     1800 non-null   object 
 2   day_type                      1800 non-null   object 
 3   shift                         1800 non-null   object 
 4   tariff_period                 1800 non-null   object 
 5   active_line_count             1800 non-null   int64  
 6   product_mix                   1800 non-null   object 
 7   production_qty                1800 non-null   int64  
 8   planned_production_qty        1800 non-null   int64  
 9   facility_power_kwh            1784 non-null   float64
 10  process_power_kwh             1800 non-null   float64
 11  hvac_power_kwh                1800 non-null   float64
 12  compressed_air_kwh            1777 non-null   float64
 13  peak_demand_kw                1800 non-null   float64
 14  ambient_temp_c                1800 non-null   float64
 15  ambient_humidity_pct          1800 non-null   float64
 16  idle_time_min                 1800 non-null   float64
 17  changeover_min                1800 non-null   float64
 18  compressed_air_pressure_bar   1800 non-null   float64
 19  power_factor_pct              1800 non-null   float64
 20  energy_intensity_kwh_unit     1674 non-null   float64
 21  peak_event_flag               1800 non-null   object 
 22  production_shortfall_qty      1800 non-null   int64  
 23  operator_action               1800 non-null   object 
dtypes: float64(12), int64(4), object(8)
memory usage: 337.6+ KB
```

```
display(df_raw.describe().T.round(2))

miss = df_raw.isna().sum()
print("[결측이 있는 변수]")
print(miss[miss > 0].to_string() if (miss > 0).any() else "없음")
```

	count	mean	std	min	25%	50%	75%	max
active_line_count	1,800.000	2.060	1.240	0.000	1.000	2.000	3.000	4.000
production_qty	1,800.000	90.830	58.940	0.000	42.000	85.000	139.000	218.000
planned_production_qty	1,800.000	97.760	60.150	-26.000	49.000	94.000	148.000	223.000
facility_power_kwh	1,784.000	58.150	6.180	36.140	53.940	58.100	62.370	77.980
process_power_kwh	1,800.000	110.060	67.440	0.000	53.020	105.500	164.340	273.100
hvac_power_kwh	1,800.000	54.520	21.340	22.840	35.470	49.360	72.370	119.460
compressed_air_kwh	1,777.000	41.650	10.760	17.180	33.450	40.820	50.000	67.520
peak_demand_kw	1,800.000	533.450	170.240	170.500	396.220	517.050	663.520	966.300
ambient_temp_c	1,800.000	27.020	5.530	12.300	22.400	27.100	31.600	40.400
ambient_humidity_pct	1,800.000	60.100	9.570	30.000	53.500	60.000	66.800	90.000
idle_time_min	1,800.000	5.180	3.720	0.000	2.180	5.000	7.700	20.900
changeover_min	1,800.000	1.780	4.620	0.000	0.000	0.000	0.000	24.300
compressed_air_pressure_bar	1,800.000	7.120	0.170	6.610	7.010	7.110	7.230	7.750
power_factor_pct	1,800.000	95.670	1.120	91.400	94.900	95.700	96.500	98.800
energy_intensity_kwh_unit	1,674.000	4.010	4.730	1.600	2.340	2.950	4.160	119.010
production_shortfall_qty	1,800.000	7.980	7.550	0.000	1.000	7.000	13.000	40.000

```
[결측이 있는 변수]
facility_power_kwh            16
compressed_air_kwh            23
energy_intensity_kwh_unit    126
```

---

4. 데이터 정제

항목	처리
계획생산량 음수 88건	**결측 처리 후 중앙값**으로 채움 (0으로 채우면 '비가동 계획'과 섞인다)
전력 결측	중앙값으로 채움
에너지 원단위 결측 126건	**그대로 둠** — 생산량 0이라 계산이 불가능한 구조적 결측

```python
df = df_raw.copy()

# ① 계획생산량 음수 → 결측 → 중앙값 (0으로 채우면 '비가동(0)'과 구분이 안 된다)
bad = df["planned_production_qty"] < 0
print("계획생산량 음수:", int(bad.sum()), "건 → 중앙값으로 대체")
df.loc[bad, "planned_production_qty"] = np.nan
df["planned_production_qty"] = df["planned_production_qty"].fillna(
    df["planned_production_qty"].median())

# ② 전력 결측 → 중앙값
for c in ["facility_power_kwh", "compressed_air_kwh"]:
    n = int(df[c].isna().sum())
    df[c] = df[c].fillna(df[c].median())
    print(f"{c} 결측 {n}건 → 중앙값으로 채움")

# ③ 에너지 원단위 결측 126건은 '생산량 0' 때문에 생긴 구조적 결측 → 채우지 않는다
print("\n에너지 원단위 결측:", int(df["energy_intensity_kwh_unit"].isna().sum()),
      "건 (생산량 0 구간) → 그대로 둠")
df["timestamp"] = pd.to_datetime(df["timestamp"])
```

```
계획생산량 음수: 88 건 → 중앙값으로 대체
facility_power_kwh 결측 16건 → 중앙값으로 채움
compressed_air_kwh 결측 23건 → 중앙값으로 채움

에너지 원단위 결측: 126 건 (생산량 0 구간) → 그대로 둠
```

---

5. 목표변수 — 찾아내고 싶은 사건은 얼마나 드문가

목표변수는 `peak_event_flag` 입니다. 피크 발생(Y) 을 **1**, 나머지를 **0** 으로 둡니다.

양성이 드물면 **"전부 음성"이라고만 답해도 정확도가 매우 높습니다.** 그래서 이 데이터에서는 **정확도를 믿으면 안 됩니다.**

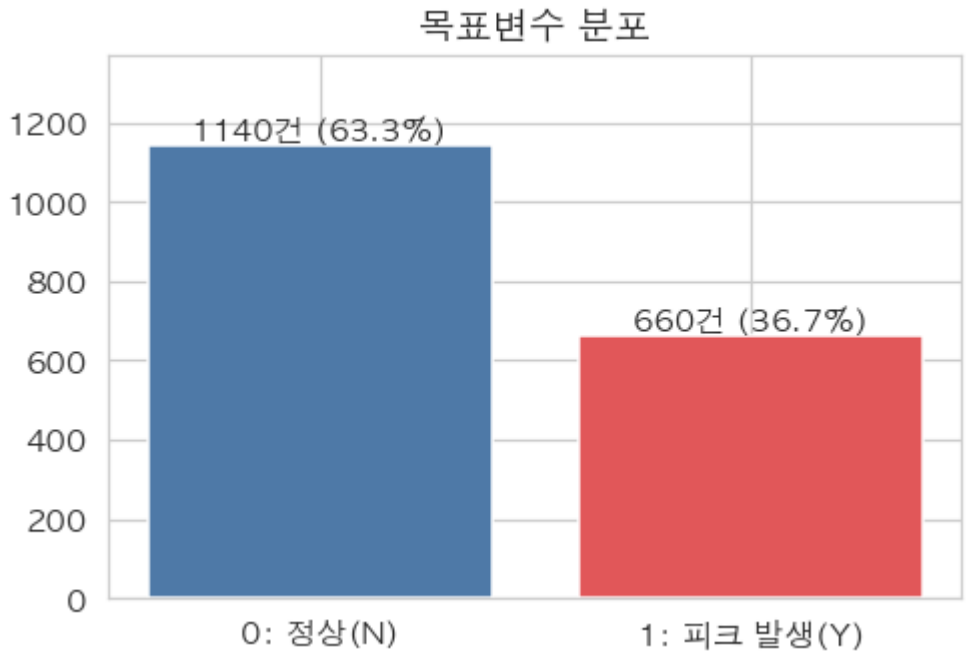
```python
y = (df['peak_event_flag'] == 'Y').astype(int)          # 1 = 피크 발생(Y)
df["target"] = y

print(df["peak_event_flag"].value_counts().to_string())
print(f"\n양성(1) 비율            : {y.mean()*100:.2f}%")
print(f"'전부 음성' 모델의 정확도 : {(1-y.mean())*100:.2f}%  ← 이 숫자를 넘어야 의미가 있다")

cnt = y.value_counts().sort_index()
plt.figure(figsize=(5, 3.2))
plt.bar(["0: 정상(N)", "1: 피크 발생(Y)"], cnt.values, color=["#4e79a7", "#e15759"])
for i, v in enumerate(cnt.values):
    plt.text(i, v, f"{v}건 ({v/len(y)*100:.1f}%)", ha="center", va="bottom")
plt.ylim(0, cnt.values.max() * 1.2)
plt.title("목표변수 분포")
plt.show()
```

```
peak_event_flag
N    1140
Y     660

양성(1) 비율            : 36.67%
'전부 음성' 모델의 정확도 : 63.33%  ← 이 숫자를 넘어야 의미가 있다
```



---

6. 탐색 ① 그룹별 양성률 — 어느 조건에서 문제가 많은가

반드시 표본 수를 함께 보세요. "10건 중 2건(20%)"과 "400건 중 40건(10%)"은 비율만 보면 앞이 2배 높지만, 앞은 1건만 달라져도 크게 흔들립니다.

```python
def group_rate(col):
    g = df.groupby(col)["target"]
    t = pd.DataFrame({"전체건수": g.size(), "양성건수": g.sum().astype(int)})
    t["양성률(%)"] = (t["양성건수"] / t["전체건수"] * 100).round(2)
    return t


GROUPS = ["active_line_count", "tariff_period", "day_type", "shift"]
for c in GROUPS:
    print(f"===== {c} =====")
    display(group_rate(c))
```

```
===== active_line_count =====
```

active_line_count	전체건수	양성건수	양성률(%)
0	164	0	0.000
1	514	0	0.000
2	488	74	15.160
3	314	266	84.710
4	320	320	100.000

```
===== tariff_period =====
```

tariff_period	전체건수	양성건수	양성률(%)
MID	674	251	37.240
OFF	756	236	31.220
PEAK	370	173	46.760

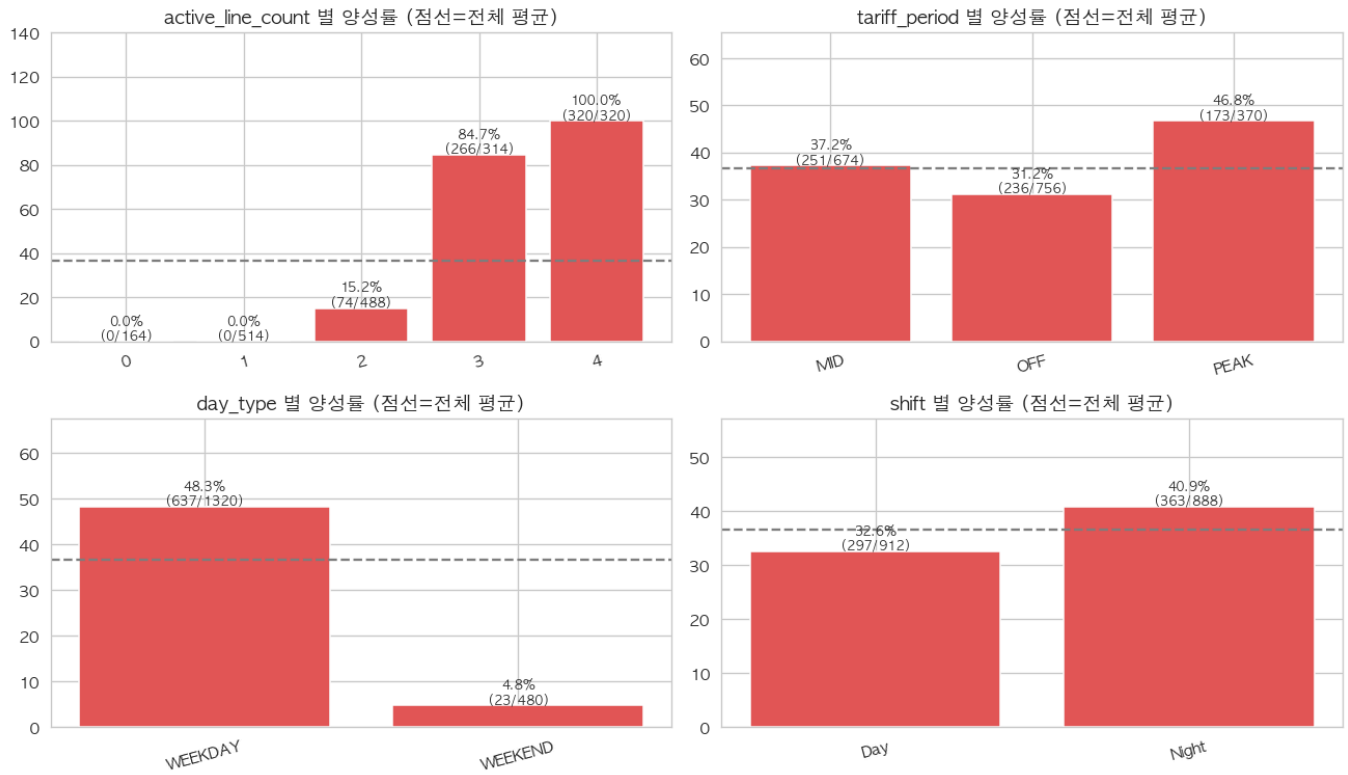
```
===== day_type =====
```

day_type	전체건수	양성건수	양성률(%)
WEEKDAY	1320	637	48.260
WEEKEND	480	23	4.790

```
===== shift =====
```

shift	전체건수	양성건수	양성률(%)
Day	912	297	32.570
Night	888	363	40.880

```
overall = df["target"].mean() * 100
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for ax, c in zip(axes.ravel(), GROUPS):
    t = group_rate(c)
    ax.bar(t.index.astype(str), t["양성률(%)"], color="#e15759")
    ax.axhline(overall, color="gray", ls="--")
    for i, (n, m, r) in enumerate(zip(t["전체건수"], t["양성건수"], t["양성률(%)"])):
        ax.text(i, r, f"{r:.1f}%\n({m}/{n})", ha="center", va="bottom", fontsize=8)
    ax.set_title(f"{c} 별 양성률 (점선=전체 평균)")
    ax.set_ylim(0, max(t["양성률(%)"].max() * 1.4, 1))
    ax.tick_params(axis="x", rotation=15)
plt.tight_layout()
plt.show()
```



---

7. 탐색 ② 교차표 — 두 조건을 겹쳐서 보기

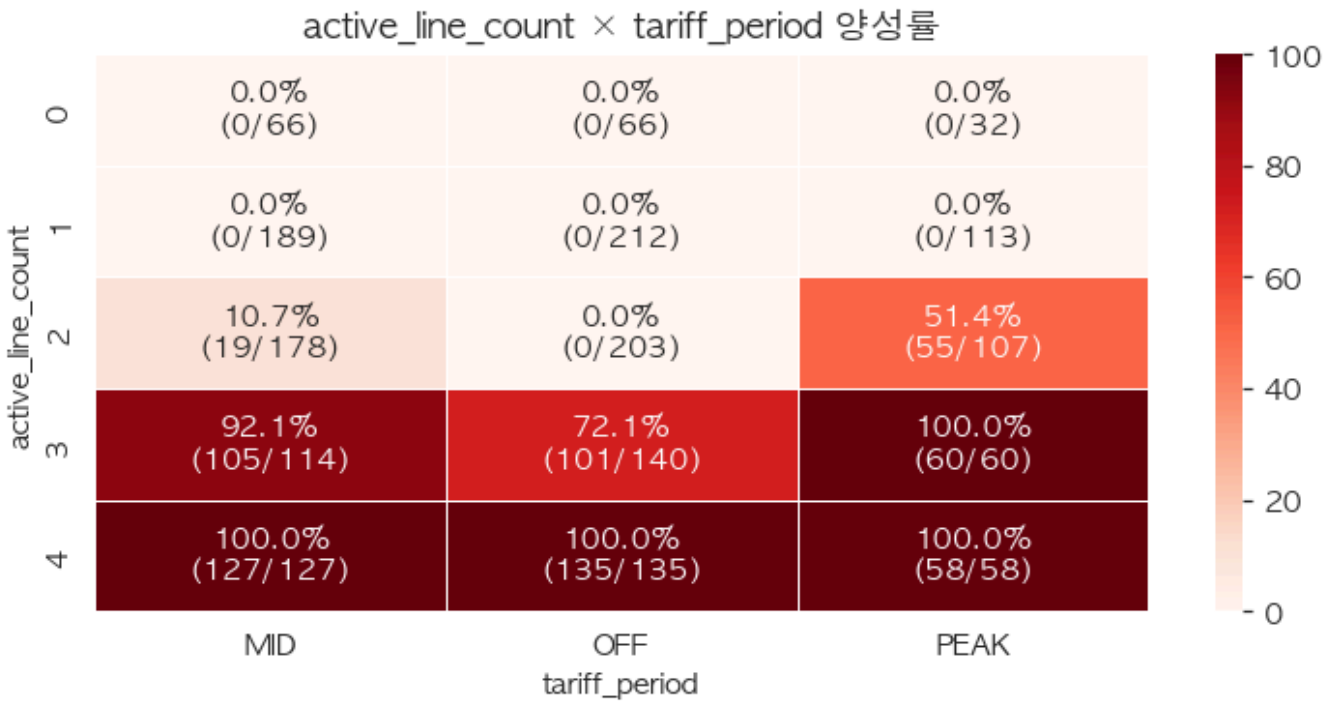
그룹을 하나씩 볼 때 보이는 차이가, 사실은 **다른 변수 때문**일 수 있습니다(**교란**). `active_line_count` 와 `tariff_period` 를 겹쳐서 확인합니다.

```
GX, GY = "active_line_count", "tariff_period"

cnt = pd.crosstab(df[GX], df[GY])
pos = pd.crosstab(df[GX], df[GY], values=df["target"], aggfunc="sum").fillna(0)
rate = (pos / cnt * 100).round(1)

plt.figure(figsize=(8, 3.4))
annot = rate.astype(str) + "%\n(" + pos.astype(int).astype(str) + "/" + cnt.astype(str) + ")"
sns.heatmap(rate, annot=annot, fmt="", cmap="Reds", linewidths=0.5)
plt.title(f"{GX} × {GY} 양성률")
plt.show()

print("칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.")
```



칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.

---

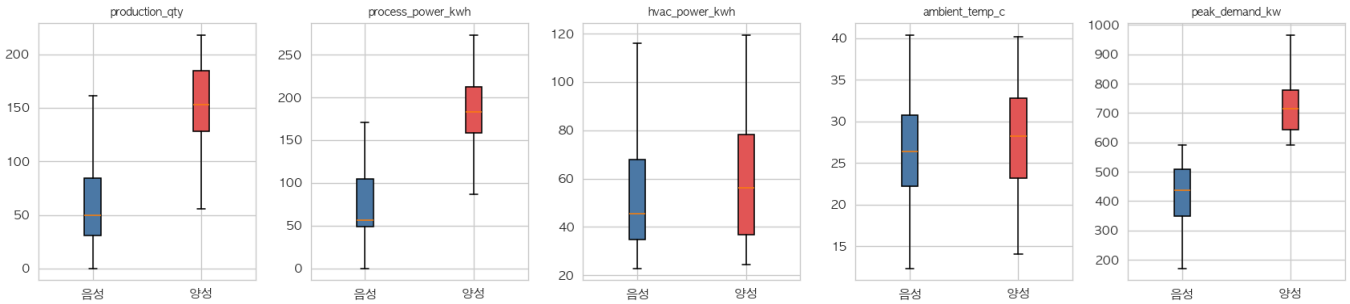
8. 탐색 ③ 수치형 변수 비교 — 양성/음성은 무엇이 다른가

상자가 많이 겹치면 **그 변수 하나로는 구분할 수 없다**는 뜻입니다. 제조 데이터는 대부분 이렇고, 그래서 **여러 변수를 조합하는 머신러닝**이 필요합니다.

```
NUM_VIZ = [c for c in [
    "production_qty", "process_power_kwh", "hvac_power_kwh", "ambient_temp_c",
    "peak_demand_kw"
] if c in df.columns]

fig, axes = plt.subplots(1, len(NUM_VIZ), figsize=(3.1 * len(NUM_VIZ), 3.6))
for ax, c in zip(np.atleast_1d(axes).ravel(), NUM_VIZ):
    data = [df.loc[df["target"] == 0, c].dropna(), df.loc[df["target"] == 1, c].dropna()]
    bp = ax.boxplot(data, patch_artist=True)
    for patch, color in zip(bp["boxes"], ["#4e79a7", "#e15759"]):
        patch.set_facecolor(color)
    ax.set_xticks([1, 2])
    ax.set_xticklabels(["음성", "양성"])
    ax.set_title(c, fontsize=9)
plt.tight_layout()
plt.show()

cmp = pd.DataFrame({"음성 평균": df[df["target"] == 0][NUM_VIZ].mean(),
                    "양성 평균": df[df["target"] == 1][NUM_VIZ].mean()})
cmp["차이"] = cmp["양성 평균"] - cmp["음성 평균"]
display(cmp.round(3))
```



	음성 평균	양성 평균	차이
production_qty	55.640	151.624	95.984
process_power_kwh	68.446	181.948	113.502
hvac_power_kwh	52.097	58.712	6.615
ambient_temp_c	26.464	27.976	1.512
peak_demand_kw	425.772	719.442	293.669

---

9. 탐색 ④ 산점도와 상관관계

산점도는 점 하나가 기록 하나입니다. 색으로 양성/음성을 구분하면 **문제가 어느 영역에 모여 있는지** 눈으로 확인할 수 있습니다.

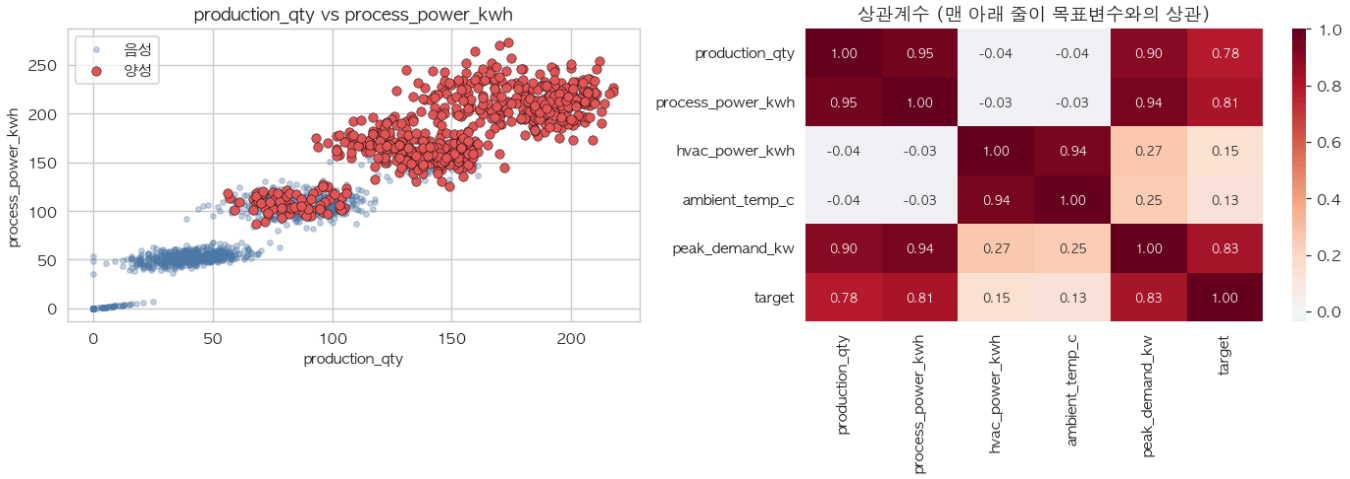
> 생산량과 공정전력의 관계 - 기울기가 '1대당 전력', 절편이 '생산과 무관한 기본 전력'

상관계수는 **직선 관계의 강도**(-1~+1)입니다. 세 가지를 기억하세요: 곡선 관계는 못 잡고, **상관은 인과가 아니며**, 이상치에 민감합니다.

```
X1, Y1 = "production_qty", "process_power_kwh"

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
neg, pos_ = df[df["target"] == 0], df[df["target"] == 1]
axes[0].scatter(neg[X1], neg[Y1], s=12, alpha=0.3, c="#4e79a7", label="음성")
axes[0].scatter(pos_[X1], pos_[Y1], s=38, c="#e15759", edgecolors="black",
                linewidths=0.3, label="양성")
axes[0].set_xlabel(X1)
axes[0].set_ylabel(Y1)
axes[0].set_title(f"{X1} vs {Y1}")
axes[0].legend()

corr = df[NUM_VIZ + ["target"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=axes[1],
            annot_kws={"size": 8})
axes[1].set_title("상관계수 (맨 아래 줄이 목표변수와의 상관)")
plt.tight_layout()
plt.show()
```



---

10. 입력변수 고르기 — 데이터 누출 막기

판단 시점(다음 구간(또는 다음 날) 생산계획을 확정하는 시점 (계획수량·가동라인·요금시간대·기상예보는 알고, 실제 전력은 아직 모름))에 **이미 알 수 있는 변수만** 사용합니다.

쓸 수 없는 변수 (결과가 나온 뒤에 생기는 값)

변수	이유
`peak_demand_kw`	그 구간의 최대수요전력. 피크 판정의 기준값 자체이므로 정답을 그대로 알려 주는 것과 같다
`operator_action`	피크를 확인한 뒤 담당자가 취한 조치. 결과에 대한 반응이다
`production_shortfall_qty`	생산부족량. 구간이 끝나야 계산된다
`energy_intensity_kwh_unit`	에너지 원단위. 실제 생산량과 실제 전력으로 나눈 사후 KPI다
`record_id`	단순 식별번호

> 성능이 비현실적으로 높으면(정확도 99%, AUC 0.99) **먼저 누출을 의심하세요.** 실무에서 가장 흔한 실패 원인입니다.

```
CAT = [c for c in ["day_type", "shift", "tariff_period", "product_mix"] if c in df.columns]
NUM = [c for c in [
    "active_line_count", "planned_production_qty", "ambient_temp_c",
    "ambient_humidity_pct", "changeover_min"
] if c in df.columns]      # 수치형 입력
LEAK = [
    "peak_demand_kw", "operator_action", "production_shortfall_qty",
    "energy_intensity_kwh_unit", "record_id"
]                                    # 사용 금지(데이터 누출)

print(f"입력 변수: 범주형 {len(CAT)}개 + 수치형 {len(NUM)}개 = {len(CAT)+len(NUM)}개")
print("범주형:", CAT)
print("수치형:", NUM)
```

```
입력 변수: 범주형 4개 + 수치형 5개 = 9개
범주형: ['day_type', 'shift', 'tariff_period', 'product_mix']
수치형: ['active_line_count', 'planned_production_qty', 'ambient_temp_c', 'ambient_humidity_pc
t', 'changeover_min']
```

---

11. 전처리 — 원핫인코딩 · 분리 · 스케일링

처리	이유
원핫인코딩	모델은 문자를 모른다. 범주마다 0/1 열을 만든다
학습/평가 분리	학습에 쓴 데이터로 평가하면 항상 성적이 좋다(시험문제를 미리 본 셈)
스케일링	변수마다 단위·범위가 달라서 큰 숫자 변수가 과대평가된다

순서가 중요합니다. 스케일러는 **학습 데이터에서만** 기준(평균·표준편차)을 배우고, 평가 데이터에는 그 기준을 **적용만** 해야 합니다. 전체 데이터로 먼저 스케일링하면 평가 데이터의 정보가 학습에 새어 들어갑니다(**데이터 누출**).

```
# ① 원핫인코딩 + 입력행렬 만들기
X = pd.concat([df[NUM], pd.get_dummies(df[CAT])], axis=1)
print("원핫인코딩 후 열 개수:", X.shape[1])

# ② 학습 75% / 평가 25% 로 분리 (stratify=y 로 양성 비율을 양쪽에 똑같이 맞춘다)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# ③ 스케일링 – 반드시 학습 데이터에서만 fit !
sc = StandardScaler()
X_train = X_train.copy()
X_test = X_test.copy()
X_train[NUM] = sc.fit_transform(X_train[NUM])    # 학습: 기준을 배우고 변환
X_test[NUM] = sc.transform(X_test[NUM])          # 평가: 배운 기준으로 변환만

print(f"학습 {len(X_train)}건 (양성 {int(y_train.sum())}) / "
      f"평가 {len(X_test)}건 (양성 {int(y_test.sum())})")
X_train.head(3)
```

```
원핫인코딩 후 열 개수: 15
학습 1350건 (양성 495) / 평가 450건 (양성 165)
```

	active_line_count	planned_production_qty	ambient_temp_c	ambient_humidity_pct	changeover_min	d
1735	1.547	1.246	-1.607	-0.662	2.380	
248	-0.065	0.070	-0.428	-1.834	-0.383	
685	-0.871	-0.928	-1.425	-1.233	-0.383	

---

12. 모델 3종 학습과 성능평가

모델	방식	특징
로지스틱 회귀	변수에 가중치를 곱해 더한 뒤 확률로 변환	해석이 쉽다
의사결정나무	스무고개처럼 질문을 이어가며 나눔	그림으로 볼 수 있다
랜덤포레스트	나무 300그루의 다수결	성능이 가장 좋은 편

`class_weight="balanced"` **를 반드시 넣으세요.** 양성이 드물면 모델은 "전부 음성"이라고 답하는 것이 손해가 적다고 판단합니다. 이 옵션은 **드문 쪽을 틀렸을 때 벌점을 크게** 주어 그것을 막습니다.

혼동행렬과 지표

	예측: 음성	예측: 양성
실제 음성	TN	**FP (헛경보)**
실제 양성	**FN (놓침)**	TP

- **Recall** = TP/(TP+FN) : 실제 양성 중 몇 %를 잡았나 → **놓치면 안 될 때 중요**
- **Precision** = TP/(TP+FP) : 경보 중 몇 %가 진짜였나 → **헛경보 비용이 클 때 중요**
- **ROC-AUC** : 임계값과 무관하게 "양성에 더 높은 점수를 줄 확률". 0.5=무작위

```python
models = {
    "로지스틱회귀": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "의사결정나무": DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                                          class_weight="balanced", random_state=42),
    "랜덤포레스트": RandomForestClassifier(n_estimators=300, max_depth=8,
                                          min_samples_leaf=5, class_weight="balanced",
                                          random_state=42),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    proba = m.predict_proba(X_test)[:, 1]
    pred = m.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"모델": name, "TP": tp, "FN": fn, "FP": fp, "TN": tn,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0),
                 "ROC_AUC": roc_auc_score(y_test, proba)})
    print(f"===== {name} =====")
    print(classification_report(y_test, pred, zero_division=0))

result = pd.DataFrame(rows).set_index("모델")
display(result.round(3))

base = 1 - y_test.mean()
print(f"참고: '전부 음성' 모델의 정확도 = {base*100:.1f}% (이 숫자는 Recall 0 이다)")
```

```
===== 로지스틱회귀 =====
              precision    recall  f1-score   support

           0       0.95      0.96      0.96       285
           1       0.94      0.91      0.92       165

    accuracy                           0.94       450
   macro avg       0.94      0.94      0.94       450
weighted avg       0.94      0.94      0.94       450

===== 의사결정나무 =====
              precision    recall  f1-score   support

           0       0.96      0.95      0.96       285
           1       0.92      0.93      0.92       165

    accuracy                           0.94       450
   macro avg       0.94      0.94      0.94       450
weighted avg       0.94      0.94      0.94       450

===== 랜덤포레스트 =====
              precision    recall  f1-score   support

           0       0.96      0.96      0.96       285
           1       0.93      0.93      0.93       165

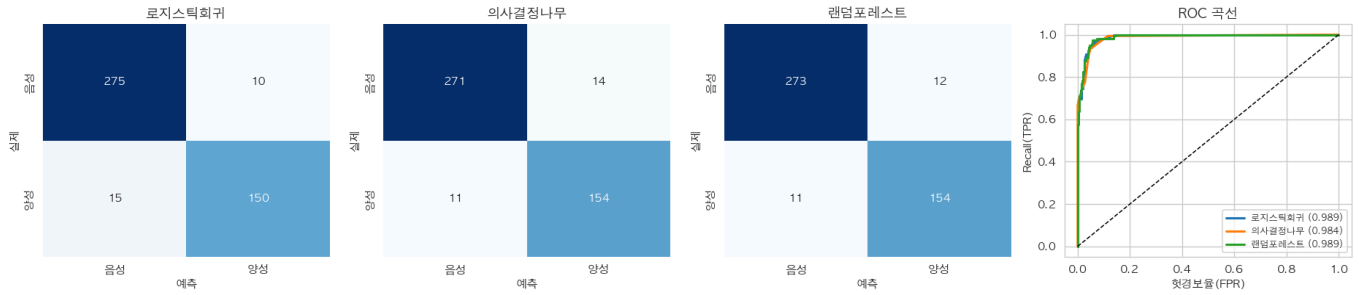
    accuracy                           0.95       450
   macro avg       0.94      0.95      0.95       450
weighted avg       0.95      0.95      0.95       450
```

모델	TP	FN	FP	TN	Recall	Precision	ROC_AUC
로지스틱회귀	150	15	10	275	0.909	0.938	0.989
의사결정나무	154	11	14	271	0.933	0.917	0.984
랜덤포레스트	154	11	12	273	0.933	0.928	0.989

참고: '전부 음성' 모델의 정확도 = 63.3% (이 숫자는 Recall 0 이다)

```
# 혼동행렬 + ROC 곡선
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8))
for ax, (name, m) in zip(axes[:3], models.items()):
    cm = confusion_matrix(y_test, m.predict(X_test), labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["음성", "양성"], yticklabels=["음성", "양성"])
    ax.set_title(name)
    ax.set_xlabel("예측")
    ax.set_ylabel("실제")

for name, m in models.items():
    proba = m.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, proba)
    axes[3].plot(fpr, tpr, lw=2, label=f"{name} ({roc_auc_score(y_test, proba):.3f})")
axes[3].plot([0, 1], [0, 1], "k--", lw=1)
axes[3].set_xlabel("헛경보율(FPR)")
axes[3].set_ylabel("Recall(TPR)")
axes[3].set_title("ROC 곡선")
axes[3].legend(fontsize=8)
plt.tight_layout()
plt.show()
```



---

13. 임계값 조정 — 모델을 바꾸지 않고 운영점 바꾸기

모델은 0/1이 아니라 **확률**을 출력합니다. `predict()` 는 0.5를 기준으로 자르는데, 이 기준은 **바꿀 수 있습니다.**

- 임계값을 **낮추면**: 경보가 늘고 → Recall↑, Precision↓
- 임계값을 **높이면**: 경보가 줄고 → Recall↓, Precision↑

현장 제약을 먼저 정하고 임계값을 고르는 것이 올바른 순서입니다.

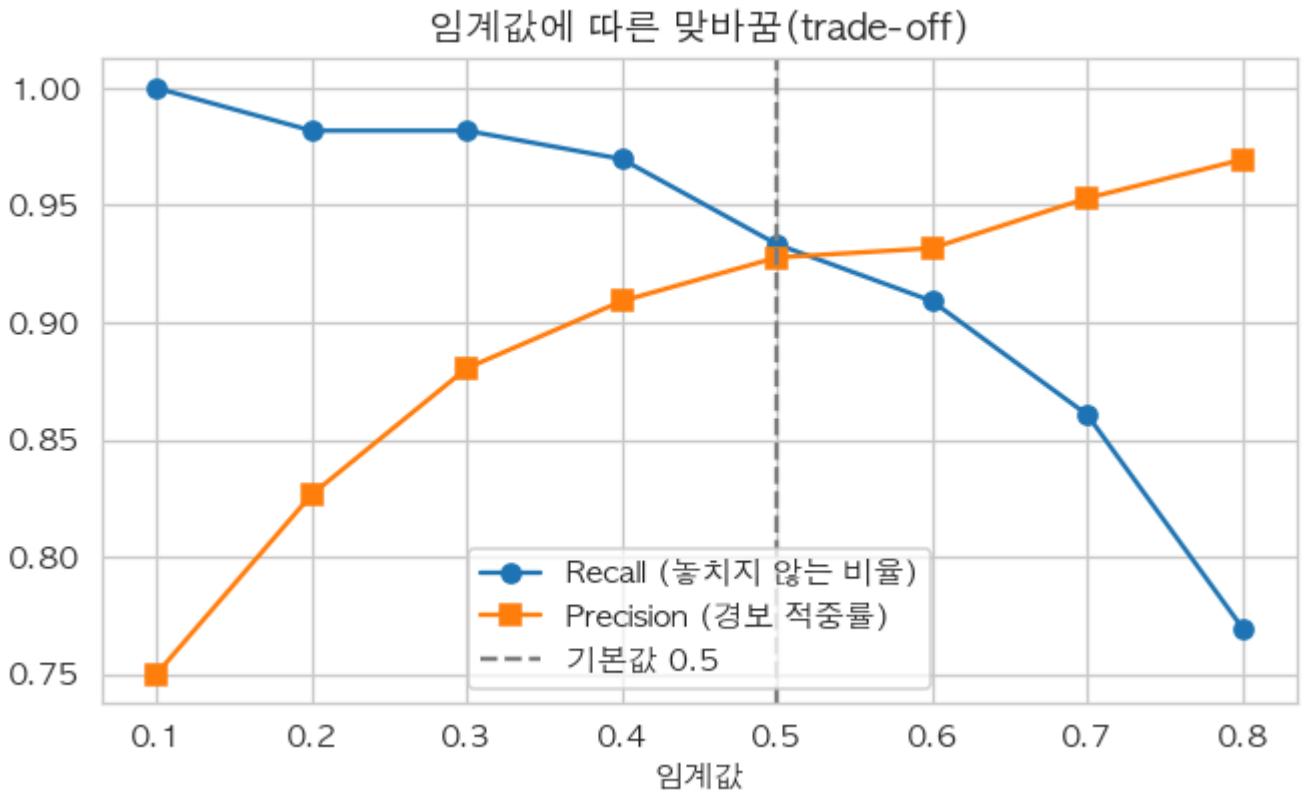
```python
best_name = result["ROC_AUC"].idxmax()          # 임계값과 무관한 지표로 기준 모델 선택
best = models[best_name]
proba = best.predict_proba(X_test)[:, 1]
print("기준 모델:", best_name)

rows = []
for th in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    pred = (proba >= th).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"임계값": th, "경보건수": int(pred.sum()), "TP(잡음)": tp,
                 "FN(놓침)": fn, "FP(헛경보)": fp,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0)})
th_tbl = pd.DataFrame(rows)
display(th_tbl.round(3))

plt.figure(figsize=(7, 3.8))
plt.plot(th_tbl["임계값"], th_tbl["Recall"], "o-", label="Recall (놓치지 않는 비율)")
plt.plot(th_tbl["임계값"], th_tbl["Precision"], "s-", label="Precision (경보 적중률)")
plt.axvline(0.5, color="gray", ls="--", label="기본값 0.5")
plt.xlabel("임계값")
plt.legend()
plt.title("임계값에 따른 맞바꿈(trade-off)")
plt.show()
```

```
기준 모델: 랜덤포레스트
```

	임계값	경보건수	TP(잡음)	FN(놓침)	FP(헛경보)	Recall	Precision
0	0.100	220	165	0	55	1.000	0.750
1	0.200	196	162	3	34	0.982	0.827
2	0.300	184	162	3	22	0.982	0.880
3	0.400	176	160	5	16	0.970	0.909
4	0.500	166	154	11	12	0.933	0.928
5	0.600	161	150	15	11	0.909	0.932
6	0.700	149	142	23	7	0.861	0.953
7	0.800	131	127	38	4	0.770	0.969



---

14. 특성중요도 — 모델은 무엇을 보고 판단했나

랜덤포레스트는 "이 변수로 나눴을 때 양성/음성이 얼마나 깔끔하게 갈리는가"를 기준으로 분기합니다. 그 기여도를 모두 합한 것이 **특성중요도**입니다(합 = 1).

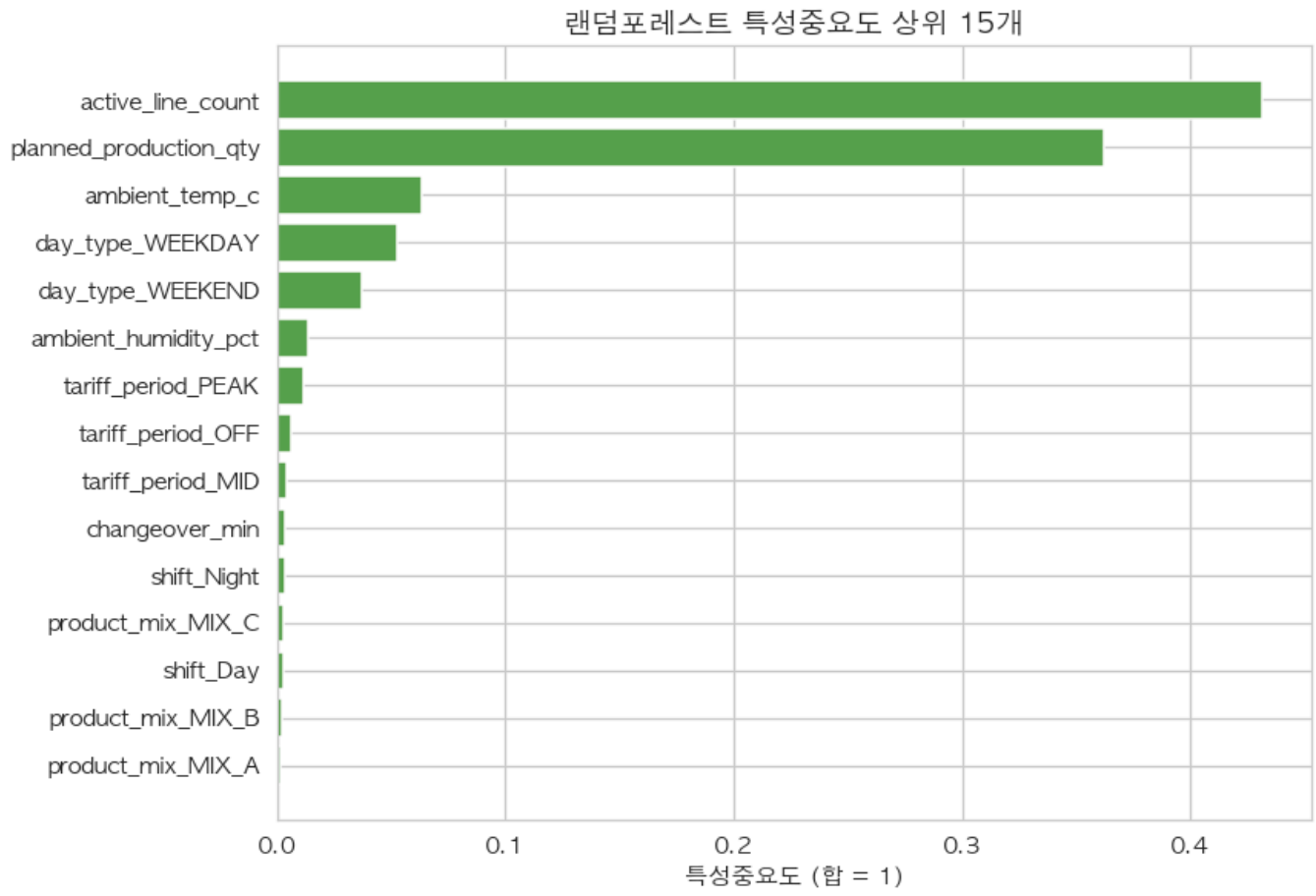
> **주의**: 특성중요도는 **"모델이 많이 참고한 변수"** 이지 **"현실의 원인"** 이 아닙니다. 인과를 주장하려면 현장에서 조건을 바꿔 보는 확인이 필요합니다.

```
rf = models["랜덤포레스트"]
imp = pd.DataFrame({"변수": X_train.columns,
                    "중요도": rf.feature_importances_}).sort_values("중요도", ascending=False)
display(imp.head(10).round(4))

top = imp.head(15)[::-1]
plt.figure(figsize=(8, 5.5))
plt.barh(top["변수"], top["중요도"], color="#59a14f")
plt.xlabel("특성중요도 (합 = 1)")
plt.title("랜덤포레스트 특성중요도 상위 15개")
plt.tight_layout()
plt.show()

print("상위 5개 변수가 전체 중요도의", f"{imp.head(5)['중요도'].sum()*100:.0f}% 를 차지합니다.")
```

	변수	중요도
0	active_line_count	0.431
1	planned_production_qty	0.362
2	ambient_temp_c	0.064
5	day_type_WEEKDAY	0.053
6	day_type_WEEKEND	0.037
3	ambient_humidity_pct	0.014
11	tariff_period_PEAK	0.011
10	tariff_period_OFF	0.006
9	tariff_period_MID	0.004
4	changeover_min	0.004



상위 5개 변수가 전체 중요도의 95% 를 차지합니다.

---

15. 정리

이 데이터에서 확인한 것

- 가동라인 수가 피크를 거의 결정한다: **0~1개 라인에서는 피크가 한 건도 없고, 4개 라인에서는 320건 전부 피크**다. 2개에서 74/488, 3개에서 266/314 발생. → "라인 4개 동시 가동"은 그 자체가 피크 조건이다.
- 피크 구간의 평균 최대수요전력은 약 719kW, 비피크는 약 426kW다.
- 평일 피크 637건 vs 주말 피크 23건. **피크는 평일 문제**다.
- 외기온도가 오르면 공조전력이 함께 오른다 → 여름철 피크에는 공조 기여가 크다.
- 에너지 원단위 평균 4.01 / 최댓값 119.01 → **분모 효과**. 생산량 0인 126건은 구조적 결측.
- 계획생산량 음수 88건 → 물리적으로 불가능. 처리 규칙을 정해야 한다.
- 교체시간(changeover)은 대부분 0이고 일부 구간만 값이 있다. IQR로 보면 이상치로 표시되지만 **실제 품종교체 작업**이다. 통계적 이상치 ≠ 오류.

결론과 다음 단계

- 가동라인 수가 피크를 거의 결정하므로 성능이 높게 나온다. **쉬운 문제**이지, 모델이 뛰어난 것이 아니다.
- 따라서 팀2의 진짜 과제는 예측 정확도가 아니라 **"생산을 지키면서 몇 kW를 옮길 것인가"** 다. 애매한 구간(가동라인 2~3개)에 집중하라.
- 다음 단계: 피크 **발생 여부(분류)** 대신 **최대수요전력 kW(회귀)** 를 예측해 보면 부하이동 효과를 kW 단위로 추정할 수 있다.

공통으로 기억할 5가지

#	원칙
1	**정확도만 보지 않는다** — 양성이 드물면 아무것도 안 해도 95%가 넘는다
2	**놓침(FN)과 헛경보(FP)의 비용은 다르다** — 어느 쪽이 비싼지는 현장이 정한다
3	**성능이 너무 좋으면 누출을 의심한다** — 결과 변수를 넣지 않았는지 확인
4	**임계값은 운영 정책이다** — 모델을 바꾸지 않고도 경보 수를 조절할 수 있다
5	**상관은 인과가 아니다** — 특성중요도는 원인이 아니라 단서다

연습문제 / 생각해 볼 질문

아래 문항은 정답을 맞히는 시험이 아니라, **발표자료에 쓸 문장을 만드는 연습**입니다. 코드를 조금씩 바꿔 보고, 결과를 한 문장으로 정리해 보세요.

1. `active_line_count` 별 피크 발생률 표를 만들고, '라인 3개'가 가장 애매한 구간인 이유를 설명하세요(314건 중 266건 피크).
2. 생산량이 0인 126개 구간만 따로 떼어, 그때도 전력이 얼마나 쓰이는지 확인하세요. 이 전력의 정체는 무엇일까요(기저부하)?
3. 세트 A(사전예측)와 세트 B(사후분석) 모델의 ROC-AUC 차이를 적고, 그 차이가 의미하는 바를 한 문장으로 쓰세요.
4. `peak_demand_kw`를 특성에 넣어 학습해 보세요. AUC가 1.0에 가까워지는 것을 확인하고 데이터 누출을 설명하세요.